

CoFEH: LLM-driven Feature Engineering Empowered by Collaborative Bayesian Hyperparameter Optimization

Beicheng Xu*
School of CS & Key Lab of High
Confidence Software Technologies
(MOE), Peking University
Beijing, China
beichengxu@stu.pku.edu.cn

Keyao Ding*
School of CS & Key Lab of High
Confidence Software Technologies
(MOE), Peking University
Beijing, China
maodeshi@stu.pku.edu.cn

Wei Liu
School of CS & Key Lab of High
Confidence Software Technologies
(MOE), Peking University
Beijing, China
eularioal@stu.pku.edu.cn

Yupeng Lu
School of CS & Key Lab of High
Confidence Software Technologies
(MOE), Peking University
Beijing, China
xinkelyp@pku.edu.cn

Bin Cui
School of CS & Beijing Key
Laboratory of Software and Hardware
Cooperative Artificial Intelligence
Systems, Peking University
Beijing, China
bin.cui@pku.edu.cn

Abstract

Feature Engineering (FE) is pivotal in automated machine learning (AutoML) but remains a bottleneck for traditional methods, which operate within rigid search spaces and lack domain awareness. While Large Language Models (LLMs) offer a promising alternative to generate unbounded operators with semantic reasoning, existing methods focus on isolated subtasks such as feature generation, falling short of free-form FE pipelines. Moreover, they are rarely coupled with hyperparameter optimization (HPO) of the downstream ML model, leading to greedy “FE-then-HPO” workflows that cannot capture strong FE-HPO interactions. In this paper, we present CoFEH, a collaborative framework that interleaves LLM-based FE and Bayesian HPO for robust end-to-end AutoML. CoFEH uses an LLM-driven FE optimizer powered by Tree of Thought (TOT) to explore flexible FE pipelines, a Bayesian optimization (BO) module to solve HPO, and a dynamic optimizer selector that adaptively interleaves FE and HPO steps. Crucially, we introduce a mutual conditioning mechanism that shares context between LLM and BO, enabling mutually informed decisions. Experiments show that CoFEH outperforms traditional and LLM-based baselines in both standalone FE and joint FE+HPO settings.

CCS Concepts

• **Computing methodologies** → **Search methodologies**; *Machine learning algorithms*.

Keywords

Feature Engineering, Large Language Models, Joint Optimization

* Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3817664>

ACM Reference Format:

Beicheng Xu, Keyao Ding, Wei Liu, Yupeng Lu, and Bin Cui. 2026. CoFEH: LLM-driven Feature Engineering Empowered by Collaborative Bayesian Hyperparameter Optimization. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3770855.3817664>

Resource Availability:

The source code is available at <https://doi.org/10.5281/zenodo.20323800>.
Extended version with full appendices: <https://arxiv.org/pdf/2602.09851>.

1 Introduction

The success of machine learning (ML) hinges on the synergy between data representation and model capacity [27]. Therefore, Feature Engineering (FE) serves as the cornerstone of an ML pipeline, directly influencing ML models’ performance and effectiveness. Broadly defined, FE constitutes a holistic transformation process that bridges raw data and model inputs, encompassing preprocessing (e.g., imputation, normalization), transformation (e.g., encoding, discretization), generation (synthesizing new interactions), and selection (filtering irrelevant signals) [43, 62]. However, manual FE is labor-intensive and demands deep domain expertise. To lower barriers and streamline deployment, the AutoML community has proposed several methods to automate the FE process [21, 29–31, 56, 60]. Many end-to-end AutoML systems also incorporate FE by casting it as a search problem over a predefined space of transformation operations and pipeline templates [14, 34, 36, 51]. Leveraging an optimizer like Bayesian optimization (BO), they explore finite configurations to construct FE pipelines.

Despite facilitating automation, these traditional approaches come with three fundamental drawbacks: (i) They are semantics-agnostic and depend on expensive trial-and-error without leveraging domain priors; (ii) They enforce rigid FE pipeline templates, often restricting pipelines to fixed sequences (e.g., generation followed by selection [21, 60]) or isolated sub-tasks, precluding dynamic interleaving or flexible composition; (iii) They restrict the

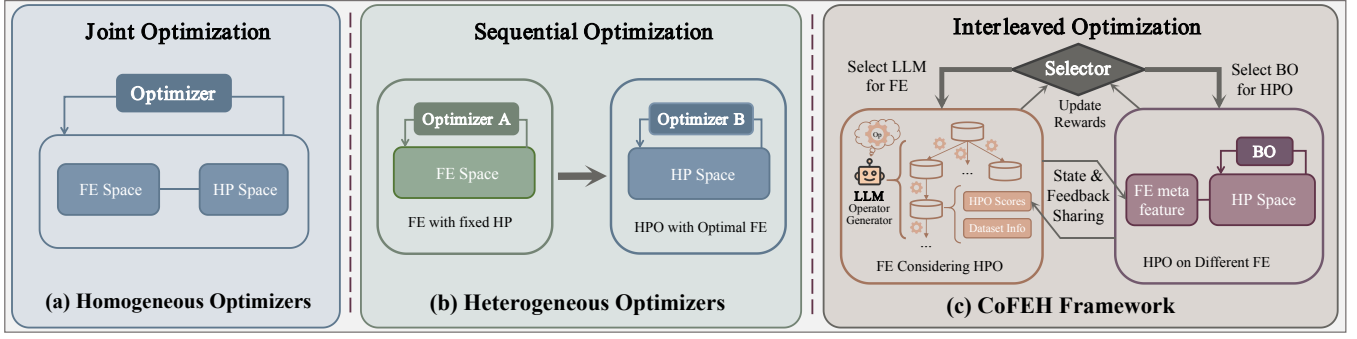


Figure 1: Comparison of optimization workflows: existing methods vs. CoFEH.

search to closed operation libraries (e.g., a small set of built-in preprocessing algorithms or basic arithmetic), which prevents discovering task-specific operations beyond predefined primitives.

To address these gaps, Large Language Models (LLMs) have been introduced to automated FE [9, 15, 53]. Through domain knowledge and code generation capabilities [6], LLMs can propose semantically grounded operations and bespoke logic beyond fixed operation libraries, alleviating drawbacks (i) and (iii) mentioned above. However, most existing methods target only a single FE subtask—most notably feature generation [1, 15, 20, 44, 61]—resulting in a homogeneous and simplistic FE pipeline. This leads to our first challenge: **C1: Free-form FE**. How can we empower LLMs to construct a truly free-form FE pipeline that flexibly composes heterogeneous operations (preprocessing, transformation, generation, and selection)?

While LLMs are inherently well-suited for FE, existing methods overlook the critical synergy with Hyperparameter Optimization (HPO) of the downstream ML model. In modern AutoML, HPO is predominantly driven by Bayesian optimization (BO) due to its sample efficiency and uncertainty modeling [36, 47, 49]. Consequently, most LLM-based FE methods that aim for end-to-end performance must attach a BO-based HPO module. Because the two optimizers are heterogeneous and operate over differently represented spaces, they typically fall back to a greedy sequential routine—first optimizing FE under a fixed model, then tuning hyperparameters on the frozen features (Fig. 1(b)) [15, 20, 37, 44]. This greedy strategy ignores the strong dependency between feature representation and model capacity. In contrast, traditional methods (e.g., Auto-sklearn [14]) naturally enable joint optimization by enforcing a homogeneous search space (Fig. 1(a)). This creates a “capability-integration paradox”: LLMs excel at FE exploration but lag behind traditional methods in FE–HPO coupling. To couple LLM-based FE with BO-based HPO, we advocate an **interleaved** optimization, which raises two further challenges: **C2: Collaborative FE–HPO optimization**. How can we jointly optimize LLM-based FE and HPO in a coupled manner, rather than optimizing each in isolation? and **C3: Task-adaptive scheduling**. How can we allocate budget between FE and HPO according to their task-specific utility?

To address these challenges, we propose CoFEH (Fig. 1(c)), a novel framework for holistic ML pipeline optimization. CoFEH is designed to unleash the full potential of LLMs in crafting free-form, human-expert-like FE pipelines, while coupling this with a state-of-the-art BO-based HPO. Our contributions are as follows:

(i) For **C1**, we propose an LLM-driven tree-of-thought FE pipeline optimizer to explore unconstrained pipeline topologies and operators, achieving truly free-form feature engineering. (ii) For **C2**, we introduce a mutual conditioning mechanism, which establishes a bidirectional information flow between the LLM and BO, enabling them to make decisions conditioned on each other to avoid isolated, sub-optimal tuning. (iii) For **C3**, we introduce a PUCB-based dynamic optimizer selector that adaptively allocates the optimization budget between FE and HPO, facilitating efficient interleaved optimization. (iv) Experiments on 28 public datasets show that CoFEH outperforms state-of-the-art traditional and LLM-based baselines in both standalone FE and end-to-end pipeline optimization.

2 Background and Motivation

2.1 The Formal Machine Learning Pipeline

The standard supervised machine learning pipeline can be formalized as a hierarchical optimization problem. Given a dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ where $\mathbf{x}_i \in \mathcal{X}$ represents the raw input space and $y_i \in \mathcal{Y}$ the target, the objective is to learn a mapping from raw data to target that minimizes the generalization error. This process is typically decomposed into two interconnected sub-problems: **Feature Engineering (FE)** [21, 29–31, 35] of data and **Hyperparameter Optimization (HPO)** [4, 13, 22, 28] of ML model.

To automate pipeline construction, traditional AutoML systems cast the entire process into a black-box optimization framework, and leverage methods like BO [14, 36, 51, 54], genetic programming [45], reinforcement learning [12], and ADMM [39] to navigate the search space. Complementing the traditional approaches, Large Language Models (LLMs) have introduced a knowledge-driven paradigm, acting as expert-like agents that propose task-adaptive feature transformation and model configurations [20, 37, 38, 41, 42, 55].

In the remainder of this section, we analyze prior work from the perspectives of FE and HPO, and discuss the key challenges and opportunities in jointly optimizing them within a unified framework.

2.2 Feature Engineering

We define FE in a broad sense, covering the entire process from raw data to the model input space. Let $\mathcal{T}$ denote the space of typed **feature operations**, encompassing data preprocessing, transformations, feature generation, and selection, etc. Consequently, an

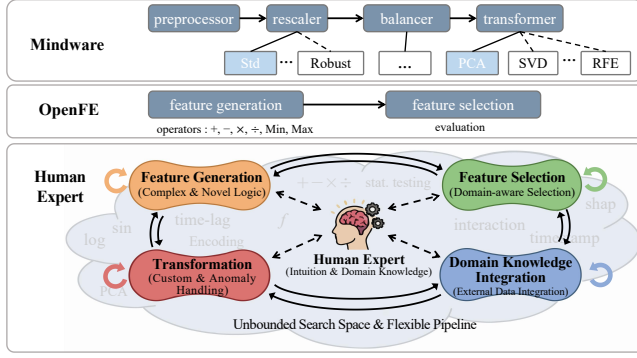


Figure 2: AutoML vs. human expert: The freedom of FE.

FE pipeline can be expressed as a composition of k operations,

$$T = t_k \circ t_{k-1} \circ \dots \circ t_1, \quad t_j \in \mathcal{T}. \quad (1)$$

The ultimate goal of this FE pipeline is to project the raw input space $\mathcal{X}$ into an optimized latent space $\mathcal{X}' = T(\mathcal{X})$ for the downstream ML model. Ideally, feature engineering is a creative and semantics-intensive process driven by domain expertise. As illustrated in the bottom panel of Fig. 2, human experts do not adhere to a rigid template. Instead, they leverage prior knowledge and intuition to navigate an unbounded search space to build a free-form pipeline.

However, such an unbounded process remains beyond the reach of traditional optimization algorithms (e.g., BO), which necessitate well-defined, finite search spaces. Consequently, existing AutoML systems typically impose severe structural constraints to make the problem tractable [14, 45, 51]. As depicted in the upper panels of Fig. 2 (e.g., MindWare [36], OpenFE [60]), these simplifications result in three limitations: (i) **Semantic agnosticism**: Lacking the capacity to incorporate domain knowledge, these systems rely exclusively on brute-force, data-driven trial-and-error. (ii) **Rigid topology**: The pipeline structures are ossified into limited and fixed sequences, preventing dynamic re-ordering or recursion. For instance, MindWare enforces a strict four-stage workflow, while OpenFE is confined to a generation-selection pipeline. (iii) **Restricted operation set**: The search is confined to a closed library of predefined algorithms or mathematical primitives (e.g., +, -, ×), precluding novel operations beyond the system’s hard-coded scope.

To overcome these bottlenecks, recent advances have proposed LLM-based FE as an alternative. This direction is part of a broader effort to apply LLMs to tabular-data tasks [11]. With extensive domain knowledge and code-generation capabilities, LLMs can emulate human-like semantic reasoning, enabling flexible feature transformations beyond closed operation libraries. This potential has sparked a wave of research across the FE spectrum, including preprocessing [8, 53], feature selection [9, 25], and feature generation [1, 15, 20, 32, 44, 61]. However, these studies typically focus on isolated sub-tasks, most notably feature generation. Consequently, they yield monolithic pipelines restricted to homogeneous operations, precluding the dynamic interleaving of diverse FE operations.

Conclusion #1: The intrinsically semantic nature of FE makes LLMs its natural architect, yet fully harnessing this power requires adopting truly free-form pipeline topologies.

2.3 Hyperparameter Optimization

Given the transformed features $\mathcal{X}'$, a learner $\mathcal{A}$ is selected to induce a model. In AutoML, this is formulated as the Combined Algorithm Selection and Hyperparameter Optimization (CASH) problem [51]. In this work, we use HPO as an umbrella term that includes CASH, where the configuration space Λ jointly encompasses both the choice of the learning algorithm (e.g., XGBoost, MLP) and its associated hyperparameters. The goal of HPO is to identify the optimal configuration λ^* that minimizes the generalization error.

The prevailing approach for solving this task is Bayesian Optimization (BO) [14, 34, 36, 47, 49, 51, 52]. BO solves black-box problems by iterating three steps [4, 24, 50]: 1) fit a surrogate f on the observed data $\mathcal{O} = \{(\lambda_i, v_i)\}_{i=1}^{n-1}$; 2) sample and select the next configuration by maximizing an acquisition function α : $\lambda_n = \arg \max_{\lambda} \alpha(\lambda; f)$; 3) evaluate λ_n to obtain the performance v_n and update $\mathcal{O} \leftarrow \mathcal{O} \cup \{(\lambda_n, v_n)\}$. BO is favored for its principled modeling of complex search spaces and acquisition-driven balance of exploration and exploitation. While recent works have tried to adapt LLMs for HPO [10, 38, 40, 59], LLM optimizers suffer from inherent limitations, lag behind BO methods, and prove unreliable in many practical scenarios [23]. They lack a surrogate model for objective and uncertainty quantification, and struggle to utilize the full optimization history due to context window limits.

Conclusion #2: BO remains the gold standard for HPO, surpassing the current reach of LLMs in hyperparameter spaces.

2.4 Joint Optimization of FE and HPO

While FE and HPO are often treated as orthogonal tasks, they are in fact strongly coupled. We formulate AutoML as an optimization problem that simultaneously searches for the optimal FE pipeline T^* and model configuration λ^* to minimize the validation loss:

$$(T^*, \lambda^*) = \arg \min_{T \in \mathcal{T}^k, \lambda \in \Lambda} \mathcal{L}_{val}(\mathcal{A}(T(\mathcal{D}_{train}); \lambda), T(\mathcal{D}_{val})), \quad (2)$$

where $\mathcal{T}^k$ represents the unbounded space of feature engineering pipelines and Λ denotes the hyperparameter space of learner $\mathcal{A}$.

Existing approaches to this problem can be categorized based on the nature of their optimizers, as visualized in Fig. 1. (i) **Homogeneous optimizer framework** (Fig. 1(a)), employs a single type of optimizer for both tasks, which naturally enables a joint optimization. Traditional AutoML systems, such as TPOT [45], Auto-sklearn [14], and Mindware [36], integrate FE and HPO into a unified yet restricted search space, enabling joint modeling and optimization. Similarly, LLM-based frameworks like ML-Master [41] and AIDE [26] treat ML pipeline generation as a unified code synthesis task, prompting LLMs to co-propose feature transformations and model configurations within a single reasoning context. (ii) **Heterogeneous optimizer framework** (Fig. 1(b)), adopts distinct optimization strategies for FE and HPO. This scenario typically necessitates a sequential, greedy strategy: optimizing FE with a fixed learner, followed by HPO on the derived feature set. Traditional methods (e.g., AutoFeat [21], OpenFE [60]) optimize features via fixed proxy models before performing post-hoc HPO. Similarly, LLM-based frameworks such as CAAFE [20] and ELLM-FT [15] focus exclusively on feature evolution, treating HPO as an isolated downstream task. Crucially, this sequential approach suffers from

greedy myopia: it fails to capture the strong dependency between features and model hyperparameters. By fixing hyperparameters during FE, the optimizer inevitably discards potential high-value features that require specific hyperparameter tuning to perform well, thereby trapping the system in sub-optimal local minima.

Therefore, although Conclusions #1 and #2 suggest that LLMs are better suited for FE while BO is more effective for HPO, jointly optimizing them is non-trivial due to their inherent heterogeneity. Instead of a greedy sequential optimization, we argue for an **inter-leaved optimization** strategy that continuously evaluates different (FE pipeline, model configuration) combinations to avoid poor local optima. This raises two key challenges: (i) **mutual conditioning and shared context**—the LLM must refine FE with awareness of how features interact with different configurations, and BO must model hyperparameters conditioned on the FE pipeline. Otherwise, if the LLM evaluates feature quality without distinguishing the underlying configurations (and vice versa for BO), the historical performance data becomes noisy and misleading, obscuring the true causal impact of any single modification. (ii) **adaptive resource allocation**—the system must decide when to invest budget in improving FE versus HPO, since their marginal utility varies substantially across tasks (e.g., some datasets are feature-sensitive while HPO yields limited gains, and vice versa).

Conclusion #3: Effective AutoML requires interleaving LLM-based FE and BO-based HPO, with mutual conditioning and task-adaptive budget allocation.

3 Method

In this section, we present CoFEH, a framework for LLM-driven Feature Engineering empowered by Collaborative Hyperparameter optimization. It comprises three core components: (i) an LLM-based Tree-of-Thought search for FE pipeline; (ii) a mutual conditioning mechanism that enables collaborative tuning between the LLM-driven FE and BO-based HPO; and (iii) a dynamic optimizer selector to adaptively allocate resources across the optimization process.

3.1 LLM-based Feature Engineering

To effectively navigate the unbounded and semantics-intensive search space of feature engineering, we reformulate the construction of the pipeline from Eq. (1) as a **sequential decision-making problem**. We define a **dataset state** s as the representation of the data at a given stage of transformation, where the initial state $s_0 = \mathcal{X}$ is the raw dataset. Each operation $t_j \in \mathcal{T}$ is viewed as an action that transitions the system from state s_{j-1} to a new state $s_j = t_j(s_{j-1})$. Consequently, the optimization of the FE pipeline is equivalent to finding an optimal sequence of **actions** $(t_1, t_2, \dots, t_k)$ that maximizes the downstream performance reward:

$$\max_{t_1, \dots, t_k} \text{Reward}(t_k \circ t_{k-1} \circ \dots \circ t_1(s_0)). \quad (3)$$

Standard linear prompting is ill-suited for this complex search space as it follows a singular reasoning path without the capacity for backtracking. To bridge this gap, we adopt the Tree of Thought (ToT) paradigm [57], enabling the LLM to explore multiple reasoning branches. As illustrated in the search tree (Fig. 3, left), intermediate dataset states (s_i) are treated as “thoughts”—discrete nodes in

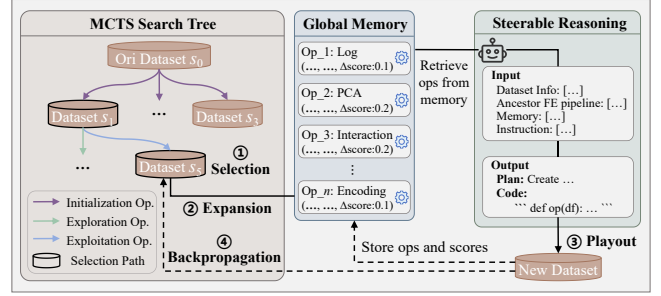


Figure 3: FE workflow of CoFEH.

a hierarchical search space connected by LLM-generated FE operations. We implement this ToT search via a Monte Carlo Tree Search (MCTS) [33], which systematically balances the discovery of novel operations with the exploitation of historical knowledge through four iterative steps (Fig. 3): 1) **Selection**: MCTS traverses from the root (s_0) along a selection path to identify the most promising dataset state for further expansion. 2) **Expansion**: the LLM is prompted to generate a new FE operation to process the dataset in the selected node. 3) **Playout**: Utilizing the executable code synthesized by the LLM, the dataset is transformed and subsequently evaluated through a downstream ML model. This process returns a validation score v_i , thereby instantiating a new dataset state s_{i+1} as a successor node in the tree. 4) **Backpropagation**: The results are propagated back to update three statistics for each ancestor node s : visit count N_s , cumulative reward R_s , and subtree best performance $v_s^{\max}$ (the maximum score observed within its subtree). We first compute a binary reward $r = \mathbb{I}(v_{new} > v_{s_0}^{\max})$, indicating whether a new global optimum has been achieved. Subsequently, we update the statistics: $N_s \leftarrow N_s + 1$, $R_s \leftarrow R_s + r$, and $v_s^{\max} \leftarrow \max(v_s^{\max}, v_{new})$.

3.1.1 Selection Down the MCTS Tree. The selection phase initiates at the root node s_0 , representing the original dataset. It traverses the search tree by recursively choosing the child node that maximizes the Upper Confidence Bound for Trees (UCT) criterion [2]:

$$s^* = \arg \max_{s' \in \text{children}(s)} \left(Q(s') + C_1 \cdot \sqrt{\frac{\ln N_s}{N_{s'}}} \right), \quad (4)$$

where C_1 is a hyperparameter balancing exploitation and exploration. The exploitation term $Q(s')$ evaluates the potential quality of the candidate feature transformation path and is defined as:

$$Q(s') = \frac{R_{s'}}{N_{s'}} + \tilde{v}_{s'}^{\max}, \quad (5)$$

where the first term $R_{s'}/N_{s'}$ represents the average reward, reflecting the historical success rate of discovering superior states along this branch. The second term $\tilde{v}_{s'}^{\max}$ denotes the normalized best validation performance observed within the subtree rooted at s' (min-max normalized using the global range of observed metric scores). This formulation of $Q(s')$ ensures that the search prioritizes paths exhibiting both high absolute performance (via $\tilde{v}^{\max}$) and strong potential for iterative improvement (via the average reward). Through this mechanism, the algorithm identifies a selection path extending from s_0 to a non-fully expanded target node, as shown by the bold-bordered nodes in the left panel of Fig. 3.

3.1.2 Expansion Through Steerable Reasoning. Upon identifying a target node s_{base} through UCT policy, the steerable reasoning module expands the search tree by synthesizing a new FE operation. We construct a structured prompt for the LLM comprising the following core components: (i) Dataset info (ψ): Semantic descriptions and statistical metadata of the base dataset to provide domain context. (ii) Ancestor FE pipeline (T_{anc}): The sequence of operations previously applied from s_0 to s_{base} , which defines the current data state. (iii) Memory ($\mathcal{M}_{\text{base}}$): A collection of high-performing operations archived from previous successful trials. Crucially, this component is provided only under the EXPLOITATION directive to facilitate refinement. (iv) Directive (d): An optimization instruction $d \in \{\text{INITIALIZATION}, \text{EXPLORATION}, \text{EXPLOITATION}\}$ that explicitly steers the LLM’s strategy. Formally, the LLM generates a reasoning chain $\mathcal{R}$ followed by the code for a new operation t_{new} :

$$(\mathcal{R}, t_{\text{new}}) = \text{LLM}(\psi, T_{\text{anc}}, \mathcal{M}_{\text{base}}, d). \quad (6)$$

The synthesized operation code is then executed to transform the base dataset, instantiating a new dataset state: $s_{\text{new}} = t_{\text{new}}(s_{\text{base}})$. Crucially, the directive d explicitly governs the search behavior:

- **INITIALIZATION** is invoked specifically for the root node s_0 to generate a high-quality initial FE operation.
- **EXPLORATION** encourages the LLM to propose novel operations to probe unknown regions of the search space.
- **EXPLOITATION** distills elite experiences and operations from $\mathcal{M}_{\text{global}}$ to refine the current dataset.

Detailed prompt templates are shown in Appendix A. The root node s_0 is treated as non-fully expanded until the INIT. directive has been executed five times. Any other node is considered fully expanded only after both EXPLORATION and EXPLOITATION directives have been performed twice. This ensures that each promising dataset state is thoroughly investigated through both creative discovery and historical refinement before the search moves deeper.

3.1.3 Global Memory and Operation Retrieval. As illustrated in the central module of Fig. 3, every historical operation t generated during the search is archived in the Global Memory $\mathcal{M}_{\text{global}}$. Each entry is stored as a tuple $(\mathcal{R}, \mathcal{F}_t, v, \Delta v)$, which includes: (i) the reasoning chain $\mathcal{R}$, (ii) the subset of features $\mathcal{F}_t$ required for the operation, which is parsed from the structured format of $\mathcal{R}$, (iii) the resulting performance score v , and (iv) the relative improvement $\Delta v = v - v_{\text{base}}$ achieved over its parent node.

To support the EXPLOITATION directive, we employ a two-stage retrieval mechanism to identify high-utility operations: 1) **Functional filtering**: We first ensure semantic compatibility by matching the required feature set $\mathcal{F}_t$ with the features currently available in state s_{base} . Only operations whose functional dependencies are fully satisfied by the current dataset are retained as valid candidates. 2) **Pareto-based selection**: From the filtered candidates, we identify high-quality operations by balancing absolute performance v and relative gain Δv . We prioritize operations with high Δv for their strong transformative potential, as well as those with high v , which represent the ability to refine performance even when the pipeline has reached well-performing regimes. Formally, we construct a Pareto frontier in the $(v, \Delta v)$ objective space and select the non-dominated operations to serve as “elite experiences” $\mathcal{M}_{\text{base}}$ within

the steerable reasoning prompt. This dual-metric selection ensures the LLM distills insights from both breakthrough transformations and marginal refinements recorded in the search history.

3.2 Collaborative Tuning with HPO

The final efficacy of an FE pipeline T is inextricably linked to the hyperparameters λ of the downstream ML model. A decoupled approach, where FE is optimized independently of HPO, often results in the greedy myopia discussed in Section 2.4—the premature rejection of promising feature transformations simply because they perform poorly under default model configurations. To address this, we propose a collaborative tuning framework that performs an interleaved exploration of the joint space of (T, λ) combinations to ensure the discovery of global optima. The core challenge of joint optimization is the bidirectional dependency: BO requires a promising feature set to optimize λ , while the LLM requires an optimized model to accurately evaluate the potential of a pipeline T . We resolve this through a **mutual conditioning mechanism**.

3.2.1 BO-based HPO Conditioned on FE. In contrast to conventional HPO that operates on a static dataset, the BO conditioned on FE must navigate a dynamic search tree containing multiple dataset states. The primary challenge lies in modeling the joint influence of the FE pipeline and configuration to determine which specific dataset state s paired with which configuration λ , yields the global optimum. To achieve this, we redefine the two core components of BO: the surrogate model and the acquisition function optimizer.

Surrogate model: Since the FE pipeline lacks an explicit, continuous search space, we utilize meta-features $\phi(s)$ to characterize the dataset state after transformations. This allows the surrogate model to map discrete tree nodes into a representative feature space. We formalize the training dataset for the surrogate model f as:

$$\mathcal{D}_{\text{BO}} = \{([\phi(s_i), \lambda_{i,j}], v_{i,j}) \mid s_i \in \mathcal{V}_{\text{tree}}, \forall j\}, \quad (7)$$

where $\mathcal{V}_{\text{tree}}$ is the set of all nodes in the MCTS tree. $\lambda_{i,j}$ and $v_{i,j}$ denote the j -th ML configuration and its score evaluated on node s_i . We employ Random Forest (RF) [24] as the surrogate model due to its support for categorical variables and computational efficiency.

Acquisition function optimizer: To identify the most promising (s, λ) pair, the optimizer explores the joint space through a hybrid sampling strategy to construct a candidate pool $\mathcal{P}_{\text{cand}}$:

- **Local search:** For each node in the tree, we perform perturbations around all its historically evaluated configurations within the hyperparameter space Λ . These perturbed configurations are paired with the corresponding node’s meta-features to capture local improvements.
- **Random search:** We perform global random sampling across the hyperparameter space Λ . Each sampled configuration is combined with the meta-features of every node in the current tree to ensure a broad exploration of the joint space.

Finally, the surrogate model predicts the performance of the combinations in the candidate pool $\mathcal{P}_{\text{cand}}$, after which an acquisition function (e.g., Expected Improvement [19]) quantifies their utility and selects the optimal combination (s^*, λ^*) . This mechanism simultaneously recommends the next ML configuration and identifies the specific dataset state s^* best positioned to leverage its potential.

3.2.2 LLM-based FE Conditioned on HPO. On the other hand, the MCTS-based FE search must be informed by the HPO process. We modify the original FE search logic to incorporate two levels of HPO-driven conditioning: (i) During the Selection phase, the maximum performance $\tilde{v}_s^{\max}$ in Eq. (5) is determined by HPO; specifically, when the BO discovers a superior validation score v^* at any given node, a localized update propagates this new performance "ceiling" back through the node's ancestors to prioritize that branch in future selection cycles. This enables the MCTS to identify FE pipelines that exhibit the greatest synergy with optimized ML configurations. (ii) During the Playout phase, the new child node s_{new} inherits the best-performing ML configuration from its parent for evaluation, ensuring that new feature operations are immediately evaluated at their highest possible capacity rather than under default settings.

3.3 Dynamic Optimizer Selector

While FE and HPO exhibit strong synergy, their relative contribution to performance gains varies significantly across datasets and search stages. To optimize resource allocation, we model the selection between FE and HPO as a Multi-Armed Bandit (MAB) problem. We employ a Predictor Upper Confidence Bound (PUCB) [48] policy to dynamically decide which optimizer to execute at each step m :

$$a^* = \arg \max_{a \in \{\text{FE}, \text{HPO}\}} \left(Q(a) + C_2 \cdot \omega_a(m) \frac{\sqrt{\sum_{a' \in \{\text{FE}, \text{HPO}\}} N_{a'}}}{1 + N_a} \right), \quad (8)$$

where N_a is the number of times action a has been selected. C_2 is a hyperparameter controlling exploration pressure. $\omega_a(m)$ is the time-varying prior weight for action a at iteration m . $Q(a)$ is the exploitation term, formulated as the empirical success rate of action a in achieving a performance breakthrough. Specifically, $Q(\text{FE})$ represents the proportion of trials where the generated FE operation yields a score surpassing that of its parent node. $Q(\text{HPO})$ is defined as the frequency with which HPO on a given node discovers a configuration that exceeds the node's historical best performance. **Prior weight scheduling.** We define the prior weights $\omega_a(m)$ as linear functions of the search progress m/M (M is the total budget):

- $\omega_{\text{FE}}(m) = p_1 - (p_1 - 0.5) \frac{m}{M}$: Starts at p_1 and decays to 0.5.
- $\omega_{\text{HPO}}(m) = p_2 + (0.5 - p_2) \frac{m}{M}$: Starts at p_2 and increases to 0.5.

Given $p_1 + p_2 = 1$, these prior weights can be unified using a step parameter $\delta = \frac{p_1 - 0.5}{M}$: $\omega_{\text{FE}}(m) = p_1 - \delta m$ and $\omega_{\text{HPO}}(m) = p_2 + \delta m$.

THEOREM 3.1 (BUDGET EQUILIBRIUM). *Consider the rule in Eq. (8) under a neutral reward signal ($Q(a) = \text{const}$). Let $M \in \mathbb{Z}^+$ be the total budget. If the initial bias satisfies $0.5 \leq p_1 < \frac{M+1.5}{M+3}$, the linear scheduling of ω_{FE} and ω_{HPO} , which converges to an equilibrium of 0.5 at $m = M$, guarantees a balanced budget distribution:*

$$N_{\text{FE}}(M), N_{\text{HPO}}(M) \in \left\{ \left\lfloor \frac{M}{2} \right\rfloor, \left\lceil \frac{M}{2} \right\rceil \right\}$$

The proofs are provided in Appendix B. By leveraging this PUCB-based mechanism, the framework achieves a dual advantage: (i) With prior weight, it implements a learnability warmup that prioritizes FE initially to ensure the data is "model-ready," shielding HPO from deceptive, noise-driven signals inherent in raw features. Unlike traditional decoupled methods that optimize FE and HPO sequentially, our approach facilitates a balanced co-optimization

adapted by task-specific performance gains $Q(a)$. (ii) Theorem 3.1 guarantees reaching a 5:5 budget equilibrium in neutral reward scenarios (i.e., when the exploitation term $Q(a)$ is not considered), making the actual allocation fundamentally task-governed. In summary, FE and HPO engage in a dynamic competition for computational resources, where the final allocation is adaptively steered by task-specific empirical rewards to reach the global optimum.

3.4 Algorithm Summary

CoFEH iteratively performs joint optimization in four steps: 1) The optimizer selector applies the PUCB rule to determine whether to execute FE or HPO; 2) The selected module is executed, where FE utilizes MCTS to extend the most promising feature pipeline T given the current HPO state, while HPO employs BO to identify the optimal configuration λ conditioned on all dataset states; 3) The proposed (T, λ) pair is evaluated on the validation set to obtain a score; 4) This score is returned as a reward signal to the selector for future decisions. This process repeats until the total budget M is exhausted, after which CoFEH returns the optimal pair (T^*, λ^*) .

4 Experiment

4.1 Experiment Setup

Baselines. We evaluate CoFEH against five representative baselines, spanning both traditional and LLM-based methodologies: — *Two traditional automated FE methods:* (i) **OpenFE** [60] utilizes feature boosting and two-stage pruning for automated feature generation; (ii) **Mindware** [36], an end-to-end AutoML system that automates the entire pipeline from preprocessing to HPO with BO; — *Three LLM-based FE methods:* (iii) **OCTree** [44] leverages LLMs and decision tree reasoning as linguistic feedback for iterative feature generation; (iv) **ELLM-FT** [15] integrates evolutionary strategies with LLMs to discover optimal feature transformation sequences; and (v) **LFG** [61] conducts a guided tree search over a predefined operation set to generate new features via LLMs.

Datasets and Metrics. Following OCTree [44], we incorporate 19 classification datasets benchmarked by Grinsztajn et al. [18]. This is supplemented by 9 regression datasets from OpenML [5] and Kaggle. Dataset details are provided in Appendix C.1. We use the classification error ($1 - \text{accuracy}$) for classification and *mean squared error* (MSE) for regression tasks as performance metrics.

Downstream Models. We evaluate the frameworks across three downstream model scenarios: (i) XGBoost [7], (ii) MLP [17], and (iii) CASH [51]: the framework must simultaneously select the best model algorithm (e.g., RF, MLP) and its optimal parameters. The hyperparameter spaces for all scenarios are detailed in Appendix C.2.

Basic Settings. While it takes a different amount of time to evaluate the same model on different datasets, we use the evaluation iterations as the unit of budget. We compare all methods under two settings: (i) **Standalone FE:** conduct an FE search for a maximum of 100 iterations while maintaining default downstream model hyperparameters. (ii) **Joint FE+HPO:** A total budget of 200 iterations is allocated for full-pipeline optimization. Mindware leverages SMAC [24] (a BO variant) to simultaneously navigate the unified space of FE and HPO. Other decoupled baselines (OpenFE, OCTree, ELLM-FT, and LFG) execute 100 sequential rounds of FE followed by 100 rounds of HPO on the fixed optimal FE pipeline. Critically,

Table 1: Comparison in standalone FE and joint FE+HPO scenarios. We report test error (%) for classification and test MSE for regression across 3 runs (Mean $\pm$ Std, $\downarrow$). Best and second-best results are bolded and underlined. Bottom rows show the average rank (Avg. Rank) per scenario, while Imp. (%) quantifies the relative error reduction of joint FE+HPO relative to standalone FE.

Dataset	Standalone FE						Joint FE + HPO					
	Mindware	OpenFE	OCTree	ELLM	LFG	CoFEH	Mindware	OpenFE	OCTree	ELLM	LFG	CoFEH
<i>Classification Tasks</i>												
rl	19.65 $\pm$ 0.81	19.48 $\pm$ 0.20	22.80 $\pm$ 0.38	17.84 $\pm$ 1.80	22.77 $\pm$ 0.81	<u>17.84 $\pm$ 2.30</u>	23.20 $\pm$ 0.30	19.52 $\pm$ 0.10	22.98 $\pm$ 0.40	<u>19.63 $\pm$ 1.90</u>	22.87 $\pm$ 0.84	21.35 $\pm$ 0.11
electricity	9.08 $\pm$ 0.13	8.47 $\pm$ 0.07	8.54 $\pm$ 0.10	<u>7.71 $\pm$ 0.21</u>	8.84 $\pm$ 0.24	7.31 $\pm$ 0.35	8.28 $\pm$ 0.17	7.93 $\pm$ 0.07	7.76 $\pm$ 0.31	6.79 $\pm$ 0.48	7.65 $\pm$ 0.26	<u>7.15 $\pm$ 0.14</u>
compass	23.54 $\pm$ 0.07	19.94 $\pm$ 0.17	25.34 $\pm$ 0.18	22.93 $\pm$ 0.72	22.70 $\pm$ 0.59	<u>22.25 $\pm$ 0.08</u>	23.63 $\pm$ 0.23	<u>20.79 $\pm$ 0.09</u>	23.44 $\pm$ 0.11	23.63 $\pm$ 0.75	21.76 $\pm$ 0.17	19.57 $\pm$ 0.44
wine	23.32 $\pm$ 0.25	<u>21.43 $\pm$ 0.51</u>	22.72 $\pm$ 0.80	22.19 $\pm$ 0.16	23.13 $\pm$ 0.27	19.45 $\pm$ 0.18	22.70 $\pm$ 1.10	<u>22.06 $\pm$ 0.55</u>	23.46 $\pm$ 0.91	23.54 $\pm$ 0.44	23.41 $\pm$ 0.48	19.50 $\pm$ 0.62
house_16H	12.30 $\pm$ 0.04	13.03 $\pm$ 0.18	<u>11.62 $\pm$ 0.10</u>	11.76 $\pm$ 0.27	11.60 $\pm$ 0.18	12.06 $\pm$ 0.05	11.92 $\pm$ 0.24	12.38 $\pm$ 0.23	12.18 $\pm$ 0.25	12.31 $\pm$ 0.42	<u>11.96 $\pm$ 0.19</u>	12.27 $\pm$ 0.04
Magic	13.92 $\pm$ 0.26	11.87 $\pm$ 0.23	14.21 $\pm$ 0.15	14.13 $\pm$ 0.15	14.14 $\pm$ 0.05	<u>12.86 $\pm$ 0.26</u>	<u>13.12 $\pm$ 0.36</u>	10.99 $\pm$ 0.22	14.42 $\pm$ 0.15	14.39 $\pm$ 0.28	14.24 $\pm$ 0.14	13.53 $\pm$ 0.40
higgs	28.10 $\pm$ 0.13	29.67 $\pm$ 0.09	28.22 $\pm$ 0.01	28.18 $\pm$ 0.02	<u>27.74 $\pm$ 0.07</u>	27.14 $\pm$ 0.32	<u>27.22 $\pm$ 0.06</u>	27.24 $\pm$ 0.09	27.46 $\pm$ 0.09	27.45 $\pm$ 0.09	27.31 $\pm$ 0.11	26.35 $\pm$ 0.02
jannis	20.33 $\pm$ 0.13	22.03 $\pm$ 0.19	20.99 $\pm$ 0.13	21.05 $\pm$ 0.06	21.02 $\pm$ 0.08	<u>20.68 $\pm$ 0.15</u>	19.65 $\pm$ 0.12	20.77 $\pm$ 0.23	20.62 $\pm$ 0.15	20.56 $\pm$ 0.29	20.41 $\pm$ 0.01	<u>20.34 $\pm$ 0.14</u>
credit	25.10 $\pm$ 0.36	26.21 $\pm$ 0.01	24.31 $\pm$ 0.15	24.47 $\pm$ 0.28	24.13 $\pm$ 0.29	<u>24.21 $\pm$ 0.03</u>	<u>22.59 $\pm$ 0.31</u>	22.94 $\pm$ 0.07	22.95 $\pm$ 0.09	22.92 $\pm$ 0.07	22.90 $\pm$ 0.24	22.58 $\pm$ 0.21
eye_movements	30.88 $\pm$ 1.70	36.96 $\pm$ 1.10	35.06 $\pm$ 0.68	29.84 $\pm$ 1.20	36.49 $\pm$ 1.40	<u>30.31 $\pm$ 1.50</u>	36.02 $\pm$ 1.80	35.98 $\pm$ 0.70	35.57 $\pm$ 0.79	<u>30.54 $\pm$ 1.50</u>	36.99 $\pm$ 1.40	29.46 $\pm$ 0.95
kddCup09	21.07 $\pm$ 0.47	22.69 $\pm$ 0.59	20.61 $\pm$ 0.64	20.74 $\pm$ 0.47	19.75 $\pm$ 0.63	<u>20.15 $\pm$ 0.30</u>	<u>19.51 $\pm$ 0.23</u>	21.01 $\pm$ 0.62	20.10 $\pm$ 0.54	19.99 $\pm$ 0.27	19.68 $\pm$ 0.07	19.35 $\pm$ 0.44
road-safety	21.10 $\pm$ 0.01	20.92 $\pm$ 0.09	21.94 $\pm$ 0.28	21.42 $\pm$ 0.20	<u>19.87 $\pm$ 0.68</u>	19.56 $\pm$ 0.07	20.49 $\pm$ 0.05	19.60 $\pm$ 0.23	21.04 $\pm$ 0.28	20.69 $\pm$ 0.86	19.99 $\pm$ 0.41	18.53 $\pm$ 0.28
bank-marketing	21.68 $\pm$ 0.25	22.16 $\pm$ 0.27	20.64 $\pm$ 0.23	<u>20.80 $\pm$ 0.36</u>	21.00 $\pm$ 0.11	21.38 $\pm$ 0.30	<u>19.79 $\pm$ 0.13</u>	20.55 $\pm$ 0.01	20.16 $\pm$ 0.07	20.31 $\pm$ 0.67	20.15 $\pm$ 0.14	19.72 $\pm$ 0.16
phoneme	14.30 $\pm$ 0.18	<u>13.51 $\pm$ 0.11</u>	13.88 $\pm$ 0.26	14.30 $\pm$ 0.18	14.16 $\pm$ 0.18	12.37 $\pm$ 0.30	14.16 $\pm$ 0.19	<u>13.04 $\pm$ 0.05</u>	14.06 $\pm$ 0.33	14.30 $\pm$ 0.19	14.30 $\pm$ 0.20	12.48 $\pm$ 0.10
covtype	11.64 $\pm$ 0.57	10.69 $\pm$ 0.01	13.52 $\pm$ 0.22	12.99 $\pm$ 0.04	13.75 $\pm$ 1.40	<u>11.12 $\pm$ 0.61</u>	10.07 $\pm$ 0.13	15.92 $\pm$ 11.00	10.06 $\pm$ 0.15	<u>9.98 $\pm$ 0.51</u>	10.23 $\pm$ 1.40	7.87 $\pm$ 0.23
california	9.72 $\pm$ 0.10	9.86 $\pm$ 0.08	9.47 $\pm$ 0.12	9.58 $\pm$ 0.01	<u>9.31 $\pm$ 0.08</u>	9.05 $\pm$ 0.12	<u>9.27 $\pm$ 0.10</u>	9.37 $\pm$ 0.07	9.53 $\pm$ 0.05	9.74 $\pm$ 0.11	9.35 $\pm$ 0.05	8.68 $\pm$ 0.26
kdd_ipums_la	12.32 $\pm$ 0.74	12.91 $\pm$ 0.11	<u>11.95 $\pm$ 0.16</u>	11.63 $\pm$ 0.46	12.28 $\pm$ 0.10	12.06 $\pm$ 0.24	11.18 $\pm$ 0.42	11.31 $\pm$ 0.13	11.59 $\pm$ 0.29	<u>11.10 $\pm$ 0.08</u>	11.54 $\pm$ 0.10	10.93 $\pm$ 0.17
MiniBooNE	5.89 $\pm$ 0.07	6.31 $\pm$ 0.02	6.02 $\pm$ 0.04	5.97 $\pm$ 0.07	<u>5.89 $\pm$ 0.08</u>	5.62 $\pm$ 0.05	<u>5.65 $\pm$ 0.02</u>	5.92 $\pm$ 0.02	5.82 $\pm$ 0.02	5.79 $\pm$ 0.03	5.80 $\pm$ 0.03	5.51 $\pm$ 0.04
pol	1.95 $\pm$ 0.09	1.79 $\pm$ 0.05	1.69 $\pm$ 0.04	1.82 $\pm$ 0.03	1.83 $\pm$ 0.07	<u>1.75 $\pm$ 0.19</u>	<u>1.74 $\pm$ 0.07</u>	1.85 $\pm$ 0.12	1.74 $\pm$ 0.04	1.87 $\pm$ 0.10	1.94 $\pm$ 0.07	1.42 $\pm$ 0.12
<i>Regression Tasks</i>												
airfoil_self_noise	2.73 $\pm$ 0.10	2.36 $\pm$ 0.03	2.57 $\pm$ 0.06	2.44 $\pm$ 0.10	<u>2.23 $\pm$ 0.05</u>	2.01 $\pm$ 0.17	2.33 $\pm$ 0.18	2.38 $\pm$ 0.07	2.12 $\pm$ 0.10	2.20 $\pm$ 0.06	<u>1.95 $\pm$ 0.22</u>	1.50 $\pm$ 0.10
cpu_small	11.14 $\pm$ 0.25	10.12 $\pm$ 0.07	11.16 $\pm$ 0.07	11.15 $\pm$ 0.08	11.02 $\pm$ 0.15	<u>10.97 $\pm$ 0.07</u>	8.26 $\pm$ 0.34	8.51 $\pm$ 0.34	7.85 $\pm$ 0.46	<u>7.85 $\pm$ 0.58</u>	7.97 $\pm$ 0.45	9.83 $\pm$ 0.67
diamonds ($\times 10^5$)	3.18 $\pm$ 0.01	<u>2.84 $\pm$ 0.01</u>	3.04 $\pm$ 0.04	2.98 $\pm$ 0.06	2.85 $\pm$ 0.02	2.72 $\pm$ 0.04	2.92 $\pm$ 0.01	<u>2.68 $\pm$ 0.02</u>	2.90 $\pm$ 0.02	2.87 $\pm$ 0.05	2.84 $\pm$ 0.01	2.60 $\pm$ 0.05
plasma_retinol ($\times 10^4$)	5.77 $\pm$ 0.58	5.43 $\pm$ 0.09	<u>4.54 $\pm$ 0.40</u>	4.35 $\pm$ 0.21	4.64 $\pm$ 0.04	5.83 $\pm$ 0.37	5.99 $\pm$ 0.33	4.65 $\pm$ 0.30	4.82 $\pm$ 0.04	4.76 $\pm$ 0.29	4.07 $\pm$ 0.27	<u>4.37 $\pm$ 0.30</u>
forest-fires ($\times 10^3$)	4.43 $\pm$ 0.62	4.70 $\pm$ 0.31	4.27 $\pm$ 0.11	4.29 $\pm$ 0.19	3.96 $\pm$ 0.08	<u>4.26 $\pm$ 0.02</u>	<u>3.98 $\pm$ 0.04</u>	4.00 $\pm$ 0.00	4.16 $\pm$ 0.15	4.28 $\pm$ 0.39	3.97 $\pm$ 0.02	3.98 $\pm$ 0.02
housing ($\times 10^9$)	2.23 $\pm$ 0.00	2.16 $\pm$ 0.00	2.16 $\pm$ 0.03	2.09 $\pm$ 0.00	<u>2.00 $\pm$ 0.07</u>	1.97 $\pm$ 0.06	2.06 $\pm$ 0.01	<u>1.99 $\pm$ 0.02</u>	2.10 $\pm$ 0.04	2.04 $\pm$ 0.03	1.99 $\pm$ 0.06	1.89 $\pm$ 0.01
bike ($\times 10^3$)	1.65 $\pm$ 0.01	1.67 $\pm$ 0.00	1.67 $\pm$ 0.02	1.66 $\pm$ 0.01	<u>1.62 $\pm$ 0.02</u>	1.52 $\pm$ 0.02	1.56 $\pm$ 0.01	1.60 $\pm$ 0.01	1.56 $\pm$ 0.02	<u>1.53 $\pm$ 0.01</u>	1.54 $\pm$ 0.01	1.43 $\pm$ 0.03
crab	5.12 $\pm$ 0.10	5.22 $\pm$ 0.05	<u>5.11 $\pm$ 0.15</u>	5.24 $\pm$ 0.18	5.16 $\pm$ 0.04	5.07 $\pm$ 0.07	<u>4.47 $\pm$ 0.04</u>	4.45 $\pm$ 0.02	4.88 $\pm$ 0.02	4.88 $\pm$ 0.09	4.58 $\pm$ 0.04	4.47 $\pm$ 0.02
insurance ($\times 10^7$)	2.82 $\pm$ 0.00	2.66 $\pm$ 0.00	2.72 $\pm$ 0.18	2.52 $\pm$ 0.10	<u>2.51 $\pm$ 0.05</u>	2.45 $\pm$ 0.07	1.98 $\pm$ 0.05	1.97 $\pm$ 0.04	2.01 $\pm$ 0.08	1.89 $\pm$ 0.02	<u>1.90 $\pm$ 0.01</u>	1.90 $\pm$ 0.07
Avg. Rank ($\downarrow$)	4.50	4.07	3.93	3.50	<u>3.11</u>	1.82	3.46	3.86	4.57	3.93	<u>3.39</u>	1.75
Avg. Imp. ($\uparrow$)	/	/	/	/	/	/	<u>+6.27%</u>	+4.98%	+4.96%	+3.84%	+5.13%	+7.03%

to ensure a fair comparison, they employ the same SMAC-based HPO module. In contrast, CoFEH performs *interleaved optimization* across the entire 200-round budget. Each dataset is split into training (60%), validation (20%), and test (20%) sets. Optimization is driven by validation performance, after which the optimal FE pipeline and ML configuration are refitted on the combined training and validation set and evaluated on the test set. We report the results across *three independent runs* to ensure statistical robustness. **Implementation Details.** All the baselines are implemented following their official open-source versions or original methodologies. Within CoFEH, we set the exploration constants $C_1 = C_2 = \sqrt{2}$ and configure the dynamic optimizer selector with initial prior weights $p_1 = 0.9$ and $p_2 = 0.1$, satisfying the conditions required for Theorem 3.1. Our HPO component is also modified from SMAC to be conditioned on FE for collaborative optimization. Specifically, we characterize each FE pipeline using MindWare’s meta-feature set, which consolidates and extends meta-features from prior meta-learning studies [3, 46, 58], with details provided in Appendix C.3. All LLM-based methods use gemini-2.0-flash [16] as the default backbone. Appendix D.1 shows that stronger backbones improve performance, but Gemini-2.0-flash offers the best cost-efficiency with only a remarkably small performance gap.

4.2 Main Results

We first evaluate CoFEH and five baselines using XGBoost as the downstream ML model. The results across both standalone FE and joint FE+HPO scenarios are presented in Table 1. Several key observations emerge from the data: (i) **LLM-based FE vs. traditional FE:** In the standalone FE scenario, traditional methods generally underperform compared to LLM-based approaches, with Mindware (4.52) and OpenFE (4.07) occupying the bottom average ranks. As discussed in Section 2.2, traditional frameworks are restricted to predefined, hardcoded operation sets and fixed, linear workflows. (ii) **Performance of LLM baselines:** Among LLM-based baselines, OCTree exhibits the weakest performance (Rank 3.93). This is primarily due to its linear reasoning structure, which lacks a backtracking mechanism. ELLM (3.54) and LFG (3.11) achieve better results by employing genetic optimization and Tree-of-Thought strategies, respectively. (iii) **Dominance of CoFEH in standalone FE:** CoFEH achieves a state-of-the-art Average Rank of 1.84, significantly outperforming the runner-up, LFG (3.11). A key strength lies in its ability to balance exploration and exploitation with a memory mechanism, as further validated by the ablation in Appendix D.2. Moreover, unlike other LLM baselines that focus strictly on individual feature generation, CoFEH enables truly free pipelines. The

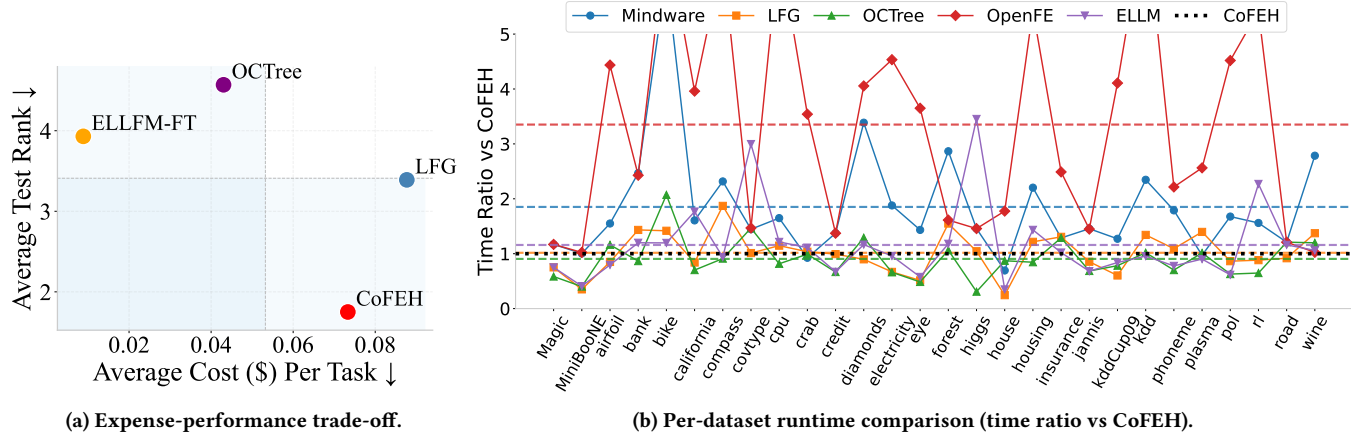


Figure 4: Cost analysis of the main experiment.

airfoil case study in Appendix D.3 further illustrates the qualitative difference. CoFEH synthesizes physically meaningful features such as a Strouhal-like number $St = f \cdot c/U$, geometry terms $\sin a$, and their interaction, then composes them with Yeo-Johnson transformation, standardization, and SelectKBest. This yields a heterogeneous pipeline spanning transformation, generation, preprocessing, and selection, whereas LLM baselines mainly remain in feature generation and OpenFE expands the feature set through many arithmetic candidates. (iv) **Sequential vs. joint optimization**: In the joint FE+HPO scenario, the relative performance of baselines shifts significantly. Decoupled sequential baselines (OpenFE, OCTree, ELLM, and LFG) exhibit lower improvement compared to standalone FE—ranging from 3.84% to 5.13%—as they freeze the FE pipeline before HPO, often trapping the system in a sub-optimal state. In contrast, Mindware achieves a notable 6.27% improvement by performing joint tuning over a unified search space. This holistic approach allows Mindware to overcome its poor FE performance (Rank 4.52), climbing to third place (Rank 3.46) in the end-to-end scenario. (v) **Superiority of CoFEH in joint tuning**: Through combining FE and HPO, CoFEH achieves a dominant improvement rate of 7.03% over standalone FE, further widening its lead with an Average Rank of 1.75. This success stems from our interleaved tuning strategy and a conditioned mechanism that facilitates mutual feedback between FE and HPO. In summary, CoFEH not only establishes a new benchmark for FE but also achieves a powerful “strong-strong” synergy in joint tuning. Appendix D.4 highlights CoFEH’s robust scalability as dataset scale increases. We further provide a Friedman test with Nemenyi post-hoc analysis in Appendix D.5 to validate the consistent superiority of CoFEH.

Cost and runtime. Fig. 4a compares the API expense and average test rank of LLM-based methods. CoFEH achieves the best cost-performance trade-off, attaining the best average rank with only \$0.07 per task; in contrast, LFG obtains the second-best rank but incurs the highest API cost, while OCTree and ELLM are cheaper but less effective. Fig. 4b further compares per-dataset end-to-end runtime normalized by CoFEH. LLM-based baselines have comparable runtimes, whereas traditional search-heavy methods are much slower: OpenFE and Mindware require on average 3.35× and 1.85× the runtime of CoFEH, respectively. This efficiency gap is

Table 2: Generalization performance (Mean $\pm$ Std) across CASH and MLP scenarios. Imp. (%) denotes the relative error reduction of CoFEH over the best baseline.

Dataset	LFG	Mindware	CoFEH	Imp. (%)
<i>CASH Scenario</i>				
pol	1.70 ± 0.15	1.87 ± 0.15	1.38 ± 0.10	+18.9%
wine	22.13 ± 0.27	20.89 ± 0.43	18.86 ± 0.39	+9.7%
airfoil_self_noise	2.72 ± 0.14	2.88 ± 0.09	1.49 ± 0.11	+45.1%
housing ($\times 10^9$)	2.12 ± 0.08	2.12 ± 0.02	1.78 ± 0.03	+16.0%
<i>MLP Scenario</i>				
pol	1.53 ± 0.08	1.42 ± 0.07	1.36 ± 0.12	+4.6%
wine	22.94 ± 0.49	22.98 ± 0.36	20.63 ± 0.21	+10.1%
airfoil_self_noise	10.49 ± 3.28	10.59 ± 2.58	6.73 ± 1.34	+35.9%
housing ($\times 10^9$)	2.70 ± 0.03	2.77 ± 0.06	2.66 ± 0.05	+1.3%
Avg. Rank ($\downarrow$)	<u>2.25</u>	2.75	1.00	–

partly due to feature explosion in traditional automated search: for example, OpenFE expands the “airfoil” dataset to 127 dimensions, whereas CoFEH and other reasoning-based methods typically keep the representation below 20 dimensions (Appendix D.3). This highlights the practical value of LLM-based FE: semantic reasoning can avoid the combinatorial accumulation of redundant features and thereby reduce the cost of repeated downstream evaluations. Averaged over the 28 datasets, evaluation dominates the end-to-end runtime, taking 1842.23s (72%), followed by LLM queries at 537.3s (21%), BO at 127s (5%), and MCTS bookkeeping at only 51.2s (2%).

4.3 Generalization to Downstream Models

While the preceding experiments demonstrate the efficacy of CoFEH using XGBoost, a robust FE framework should ideally exhibit model-agnostic generalization. In this section, we evaluate whether CoFEH can collaboratively optimize FE and HPO within two distinct downstream model types: deep learning (MLP) and multi-algorithm search (CASH). To mitigate API overhead, we select four representative datasets from the original benchmark: two classification

tasks (pol, wine) and two regression tasks (airfoil, housing). We evaluate CoFEH against the two strongest baselines in joint FE+HPO scenario from Table 1: LLM-based LFG and traditional Mindware.

The results in Table 2 demonstrate the robust generalization of CoFEH across three dimensions: (1) **Consistent superiority**: CoFEH consistently achieves the top performance across all datasets and scenarios with a perfect Average Rank of 1.00, maintaining a ranking hierarchy identical to the primary XGBoost experiments. (2) **Significant error reduction**: Compared to the strongest baseline in each task, CoFEH achieves substantial relative error reductions, most notably reaching 45.1% in the CASH scenario and 35.9% in the MLP scenario. (3) **Enhanced performance ceiling**: Compared with Table 1, the end-to-end performance in the CASH scenario generally surpasses that of XGBoost, indicating that CoFEH effectively leverages multi-algorithm search to identify models that are more compatible with the synthesized features.

4.4 Effectiveness of Collaborative Tuning

In this section, we evaluate the impact of the collaborative tuning framework, which comprises two components: the mutual conditioning mechanism for bidirectional information exchange between FE and HPO, and the dynamic optimizer selector for resource scheduling. To isolate their contributions, we conduct an ablation study comparing the full CoFEH against three variants: (i) w/o Cond, where HPO feedback and meta-feature conditioning are removed, forcing both modules to generate independent proposals that are heuristically paired with the counterpart’s current global best for evaluation; (ii) w/o Selector, which replaces dynamic scheduling with a fixed, alternating execution strategy; and (iii) Greedy, the sequential paradigm that follows the standard practice of performing 100 iterations of FE followed by 100 iterations of HPO. As illustrated in Fig. 5, which plots the average normalized best-so-far validation performance across four representative datasets, the Greedy approach yields the poorest results, confirming the suboptimality of decoupled optimization. We find that ablating either core mechanism significantly degrades convergence. Ultimately, CoFEH achieves the best Average Test Rank of 1.25, substantially outperforming the w/o Selector (2.25), w/o Cond (2.75) variants, confirming the necessity of coupled information-resource management. We next analyze the two components separately.

Effect of mutual conditioning. The w/o Cond variant removes HPO feedback and FE-state conditioning, preventing BO from distinguishing how the same hyperparameter configuration behaves under different transformed datasets. The surrogate-fitting analysis in Appendix D.6 provides direct evidence: adding FE meta-features raises the mean Spearman correlation between BO predictions and true performance from 0.587 to 0.691 across the 28 datasets, with especially large gains on FE-sensitive tasks.

Effect of dynamic optimizer selection. To further analyze the behavior of the optimizer selector, Fig. 6 examines the distribution of FE iterations across the 28 datasets in the main experiment. (i) **Temporal dynamics**: As shown in Fig. 6a, the proportion of datasets executing FE follows a clear downward trend. This aligns with our design intuition of prior weights: prioritizing structural data refinement in early stages. Notably, both optimizers remain active throughout the process, ensuring continuous co-evolution.

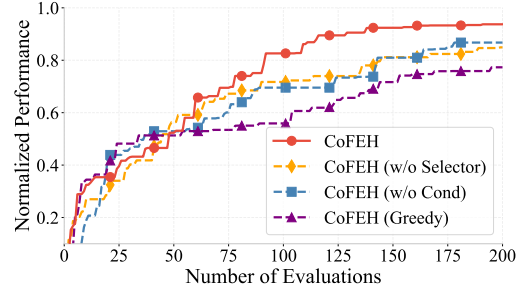
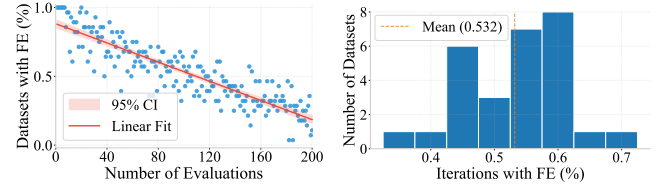


Figure 5: Ablation study of collaborative tuning.



(a) FE prop. across iterations.

(b) FE prop. across datasets.

Figure 6: FE prop. driven by the dynamic optimizer selector

(ii) **Task-adaptive allocation**: Fig. 6b illustrates the distribution of total FE iterations per dataset. While the mean proportion of 0.53 empirically substantiates the budget equilibrium predicted in Theorem 3.1, the significant variance (ranging from 0.37 to 0.7) highlights the framework’s task-adaptivity. This reflects the selector’s ability to detect whether a specific task is more sensitive to feature representations or ML configurations. Appendix D.7 provides a case study of task-adaptive allocation. We compare independent pure-FE and pure-HPO runs on two representative datasets to test whether the selector’s allocation matches each task’s dominant optimization driver. On “diamonds”, pure FE achieves a higher ceiling than pure HPO, and the selector correspondingly allocates 70% of the budget to FE. On “jannis”, pure HPO is more effective, and the selector reduces the FE share to 37%. These contrasting cases show that the selector adapts the FE/HPO budget according to the task-specific marginal utility of each optimizer.

5 Conclusion

In this paper, we presented CoFEH, a collaborative framework that bridges semantic FE discovery and numerical HPO through adaptive scheduling and mutual conditioning. Experiments across 28 public datasets show that CoFEH achieves superior performance over traditional AutoML and LLM-based baselines in both standalone FE and joint FE+HPO settings. The framework is also extensible: (i) it is agnostic to specific HPO implementations and maintains full compatibility with state-of-the-art BO variants; (ii) it enables seamless extension to image and text data via simple prompt modifications, offering a scalable foundation for future work.

Acknowledgments

This work is supported by National Natural Science Foundation of China (U23B2048, U22B2037). Bin Cui is the corresponding author.

References

- [1] Nikhil Abhyankar, Parshin Shojaei, and Chandan K Reddy. 2025. LLM-FE: Automated Feature Engineering for Tabular Data with LLMs as Evolutionary Optimizers. *arXiv preprint arXiv:2503.14434* (2025).
- [2] Peter Auer. 2002. Using confidence bounds for exploitation-exploration trade-offs. *Journal of machine learning research* 3, Nov (2002), 397–422.
- [3] Rémi Bardenet, Mártyás Brendel, Balázs Kégl, and Michele Sebag. 2013. Collaborative hyperparameter tuning. In *International conference on machine learning*. PMLR, 199–207.
- [4] James Bergstra, Rémi Bardenet, Yoshua Bengio, and Balázs Kégl. 2011. Algorithms for hyper-parameter optimization. *Advances in neural information processing systems* 24 (2011).
- [5] Bernd Bischl, Giuseppe Casalicchio, Taniya Das, Matthias Feurer, Sebastian Fischer, Pieter Gijsbers, Subhaditya Mukherjee, Andreas C Müller, László Németh, Luis Oala, et al. 2025. OpenML: Insights from 10 years and more than a thousand papers. *Patterns* 6, 7 (2025).
- [6] Li-Guo Chen, Zheng Xiao, Yi-Jiang Xu, Rui-Chuan An, Xin Wang, Yang-Ning Li, Ying-Hui Li, Yi-Dong Wang, Zheng-Ran Zeng, Qing Gao, et al. 2025. CodeRankEval: Benchmarking and Analyzing LLM Performance for Code Ranking. *Journal of Computer Science and Technology* 40, 5 (2025), 1220–1233.
- [7] Tianqi Chen. 2016. XGBoost: A Scalable Tree Boosting System. *Cornell University* (2016).
- [8] Hyunjun Choi, Jay Moran, Nicholas Matsumoto, Miguel E Hernandez, and Jason H Moore. 2023. Aliro: an automated machine learning tool leveraging large language models. *Bioinformatics* 39, 10 (2023), btad606.
- [9] Kristy Choi, Chris Cundy, Sanjari Srivastava, and Stefano Ermon. 2022. Lm-priors: Pre-trained language models as task-specific priors. *arXiv preprint arXiv:2210.12530* (2022).
- [10] Abdoulatif Cissé, Xenophon Evangelopoulos, Vladimir V. Gusev, and Andrew I. Cooper. 2025. Language-based Bayesian optimization research assistant (BORA). In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (Montreal, Canada) (IJCAI '25)*. Article 553, 9 pages. doi:10.24963/ijcai.2025/553
- [11] Yuyang Dong, Masafumi Oyamada, Chuan Xiao, and Haochen Zhang. 2024. On the use of large language models for table tasks. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 5518–5521.
- [12] Iddo Drori, Yamuna Krishnamurthy, Remi Rampin, Raoni DE PAULA LOURENCO, Jorge Piazentin Ono, Kyunghyun Cho, Claudio Silva, and Juliana Freire. 2021. AlphaD3M: Machine Learning Pipeline Synthesis. In *ICML AutoML Workshop*.
- [13] Stefan Falkner, Aaron Klein, and Frank Hutter. 2018. BOHB: Robust and efficient hyperparameter optimization at scale. In *International conference on machine learning*. PMLR, 1437–1446.
- [14] Matthias Feurer, Katharina Eggenberger, Stefan Falkner, Marius Lindauer, and Frank Hutter. 2022. Auto-sklearn 2.0: Hands-free automl via meta-learning. *Journal of Machine Learning Research* 23, 261 (2022), 1–61.
- [15] Nanxu Gong, Chandan K Reddy, Wangyang Ying, Haifeng Chen, and Yanjie Fu. 2025. Evolutionary large language model for automated feature transformation. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 39. 16844–16852.
- [16] Google DeepMind. 2025. *Gemini 2.0 Flash*. <https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/2-0-flash>
- [17] Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, and Artem Babenko. 2021. Revisiting deep learning models for tabular data. *Advances in neural information processing systems* 34 (2021), 18932–18943.
- [18] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why do tree-based models still outperform deep learning on typical tabular data? *Advances in neural information processing systems* 35 (2022), 507–520.
- [19] José Miguel Hernández-Lobato, Matthew W Hoffman, and Zoubin Ghahramani. 2014. Predictive entropy search for efficient global optimization of black-box functions. In *Advances in neural information processing systems*. 918–926.
- [20] Noah Hollmann, Samuel Müller, and Frank Hutter. 2023. Large language models for automated data science: Introducing caafe for context-aware automated feature engineering. *Advances in Neural Information Processing Systems* 36 (2023), 44753–44775.
- [21] Franziska Horn, Robert Pack, and Michael Rieger. 2019. The autofeat python library for automated feature engineering and selection. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 111–120.
- [22] Yi-Qi Hu, Yang Yu, Wei-Wei Tu, Qiang Yang, Yuqiang Chen, and Wenyuan Dai. 2019. Multi-fidelity automatic hyper-parameter tuning via transfer series expansion. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 3846–3853.
- [23] Beichen Huang, Xingyu Wu, Yu Zhou, Jibin Wu, Liang Feng, Ran Cheng, and Kay Chen Tan. 2024. Exploring the true potential: Evaluating the black-box optimization capability of large language models. *arXiv preprint arXiv:2404.06290* (2024).
- [24] Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. 2011. Sequential model-based optimization for general algorithm configuration. In *International Conference on Learning and Intelligent Optimization*. Springer, 507–523.
- [25] Daniel P Jeong, Zachary C Lipton, and Pradeep Ravikumar. 2024. Llm-select: Feature selection with large language models. *arXiv preprint arXiv:2407.02694* (2024).
- [26] Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. 2025. Aide: Ai-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138* (2025).
- [27] Michael I Jordan and Tom M Mitchell. 2015. Machine learning: Trends, perspectives, and prospects. *Science* 349, 6245 (2015), 255–260.
- [28] Lara Kallab, Elio Mansour, and Richard Chbeir. 2024. Towards ML Models' Recommendations. *Data Science and Engineering* 9, 4 (2024), 409–430.
- [29] Gilad Katz, Eui Chul Richard Shin, and Dawn Song. 2016. Explorekit: Automatic feature generation and selection. In *2016 IEEE 16th international conference on data mining (ICDM)*. IEEE, 979–984.
- [30] Ambika Kaul, Saket Maheshwary, and Vikram Pudi. 2017. Autolearn—automated feature generation and selection. In *2017 IEEE International Conference on data mining (ICDM)*. IEEE, 217–226.
- [31] Udayan Khurana, Horst Samulowitz, and Deepak Turaga. 2018. Feature engineering for predictive modeling using reinforcement learning. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 32.
- [32] Jeonghyun Ko, Gyeongyun Park, Donghoon Lee, and Kyunam Lee. 2025. Ferg-llm: Feature engineering by reason generation large language models. In *Findings of the Association for Computational Linguistics: NAACL 2025*. 4211–4228.
- [33] Levente Kocsis and Csaba Szepesvári. 2006. Bandit based monte-carlo planning. In *European conference on machine learning*. Springer, 282–293.
- [34] Erin LeDell and Sebastien Poirier. 2020. H2o automl: Scalable automatic machine learning. In *Proceedings of the AutoML Workshop at ICML*, Vol. 2020. 24.
- [35] Xiao-Lin Li, Li Ma, Xiang-Dong He, and Hui Xiong. 2020. You are how you behave—spatiotemporal representation learning for college student academic achievement. *Journal of Computer Science and Technology* 35, 2 (2020), 353–367.
- [36] Yang Li, Jiawei Jiang, Jinyang Gao, Yingxia Shao, Ce Zhang, and Bin Cui. 2020. Efficient automatic CASH via rising bandits. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 4763–4771.
- [37] Ziming Li, Qianbo ZANG, David Ma, Jiawei Guo, Tuney Zheng, Minghao Liu, Xinyao Niu, Yue Wang, Jian Yang, Jiaheng Liu, et al. 2024. AutoKaggle: A Multi-Agent Framework for Autonomous Data Science Competitions. In *ICLR 2025 Workshop Emergent Possibilities and Challenges in Deep Learning for Code*.
- [38] Siyi Liu, Chen Gao, and Yong Li. 2024. Large language model agent for hyperparameter optimization. *arXiv preprint arXiv:2402.01881* (2024).
- [39] Sijia Liu, Parikshit Ram, Deepak Vijaykeerthy, Djallel Bouneffouf, Gregory Bramble, Horst Samulowitz, Dakuo Wang, Andrew Conn, and Alexander Gray. 2020. An ADMM based framework for automl pipeline configuration. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 4892–4899.
- [40] Tennison Liu, Nicolás Astorga, Nabeel Seedat, and Mihaela van der Schaar. 2025. Large Language Models to Enhance Bayesian Optimization. In *The Twelfth International Conference on Learning Representations*.
- [41] Zexi Liu, Yuzhu Cai, Xinyu Zhu, Yujie Zheng, Runkun Chen, Ying Wen, Yanfeng Wang, Siheng Chen, et al. 2025. ML-Master: Towards AI-for-AI via Integration of Exploration and Reasoning. *arXiv preprint arXiv:2506.16499* (2025).
- [42] Daqin Luo, Chengjian Feng, Yuxuan Nong, and Yiqing Shen. 2024. Autom3L: An automated multimodal machine learning framework with large language models. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 8586–8594.
- [43] Alhassan Mumuni and Fuseini Mumuni. 2025. Automated data processing and feature engineering for deep learning and big data applications: a survey. *Journal of Information and Intelligence* 3, 2 (2025), 113–153.
- [44] Jaehyun Nam, Kyuyoung Kim, Seunghyuk Oh, Jihoon Tack, Jaehyung Kim, and Jinwoo Shin. 2024. Optimized feature generation for tabular data via llms with decision tree reasoning. *Advances in Neural Information Processing Systems* 37 (2024), 92352–92380.
- [45] Randal S Olson and Jason H Moore. 2016. TPOT: A tree-based pipeline optimization tool for automating machine learning. In *Workshop on automatic machine learning*. PMLR, 66–74.
- [46] Bernhard Pfahringer, Hilan Bensusan, and Christophe G Giraud-Carrier. 2000. Meta-Learning by Landmarking Various Learning Algorithms. In *ICML*. 743–750.
- [47] Pranav Poduval, Sanjay Kumar Patnala, Gaurav Oberoi, Nitish Srivasatava, and Siddhartha Asthana. 2024. Cash via optimal diversity for ensemble learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2411–2419.
- [48] Christopher D Rosin. 2011. Multi-armed bandits with episode context. *Annals of Mathematics and Artificial Intelligence* 61, 3 (2011), 203–230.
- [49] Yu Shen, Yupeng Lu, Yang Li, Yaofeng Tu, Wentao Zhang, and Bin Cui. 2022. Di-vbo: diversity-aware cash for ensemble learning. *Advances in Neural Information Processing Systems* 35 (2022), 2958–2971.
- [50] Jasper Snoek, Hugo Larochelle, and Ryan P Adams. 2012. Practical bayesian optimization of machine learning algorithms. *Advances in neural information*

- processing systems 25 (2012).
- [51] Chris Thornton, Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. 2013. Auto-WEKA: Combined selection and hyperparameter optimization of classification algorithms. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*. 847–855.
 - [52] Anton Vakhrushev, Alexander Ryzhkov, Maxim Savchenko, Dmitry Simakov, Rinchin Damdinov, and Alexander Tuzhilin. 2021. Lightautoml: Automl solution for a large financial services ecosystem. *arXiv preprint arXiv:2109.01528* (2021).
 - [53] Chenglong Wang, Bongshin Lee, Steven M Drucker, Dan Marshall, and Jianfeng Gao. 2025. Data Formulator 2: Iterative Creation of Data Visualizations, with AI Transforming Data Along the Way. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–17.
 - [54] Beicheng Xu, Wei Liu, Keyao Ding, Yupeng Lu, and Bin Cui. 2026. Pseo: Optimizing post-hoc stacking ensemble through hyperparameter tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. 27188–27196.
 - [55] Jinglue Xu, Jialong Li, Zhen Liu, Nagar Anthel Venkatesh Suryanarayanan, Guoyuan Zhou, Jia Guo, Hitoshi Iba, and Kenji Tei. 2024. Large language models synergize with automated machine learning. *arXiv preprint arXiv:2405.03727* (2024).
 - [56] Quanming Yao, Mengshuo Wang, Yuqiang Chen, Wenyuan Dai, Yu-Feng Li, Wei-Wei Tu, Qiang Yang, and Yang Yu. 2018. Taking human out of learning applications: A survey on automated machine learning. *arXiv preprint arXiv:1810.13306* 31 (2018).
 - [57] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafra, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
 - [58] Dani Yogatama and Gideon Mann. 2014. Efficient transfer learning method for automatic hyperparameter tuning. In *Artificial intelligence and statistics*. PMLR, 1077–1085.
 - [59] Shujian Zhang, Chengyue Gong, Lemeng Wu, Xingchao Liu, and Mingyuan Zhou. 2023. Automl-gpt: Automatic machine learning with gpt. *arXiv preprint arXiv:2305.02499* (2023).
 - [60] Tianping Zhang, Zheyu Aqa Zhang, Zhiyuan Fan, Haoyan Luo, Fengyuan Liu, Qian Liu, Wei Cao, and Li Jian. 2023. Openfe: Automated feature generation with expert-level performance. In *International Conference on Machine Learning*. PMLR, 41880–41901.
 - [61] Xinhao Zhang, Jinghan Zhang, Banafsheh Rekabdar, Yuanchun Zhou, Pengfei Wang, and Kunpeng Liu. 2025. Dynamic and adaptive feature generation with LLM. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (Montreal, Canada) (IJCAI '25)*. Article 782, 9 pages. doi:10.24963/ijcai.2025/782
 - [62] Tommaso Zoppi, Andrea Ceccarelli, and Andrea Bondavalli. 2025. A strategy for predicting the performance of supervised and unsupervised tabular data classifiers. *Data Science and Engineering* 10, 1 (2025), 75–97.

A Prompt Design

This appendix details our prompt engineering framework, spanning the expert-persona System Prompt, the structured User Prompt, and a concrete input/output example.

A.0.1 System Prompt. The System Prompt establishes the LLM’s expert role and core mission:

System Prompt

You are an elite Machine Learning Feature Engineering (FE) Expert whose mission is to analyze data distributions and synthesize high-value pipelines by generating innovative feature operations specifically designed to maximize the predictive performance of downstream ML models.

A.0.2 Structured User Prompt. The User Prompt comprises the global task description and the four core components defined in Eq. (6), which are detailed below:

(i) **Task description** defines the objective and output format:

Template: Task Description

Given the dataset metadata, transformation history, and elite memory, your task is to synthesize novel feature engineering operations to maximize the performance of the downstream model {...}. You should provide a reasoning insight followed by executable Python code.

Output Format: You must provide your insight followed by the code in the following structure:

– Reason: (Explain why this specific transformation is beneficial for the downstream model.)

– Way: (Detail the exact columns and the specific method required for the transformation.)

– Implementation: (Provide a standalone Python function. Use the following template)

```
def your_fe_name(df):
    '''A concise description of the operator.'''
    import pandas as pd
    import numpy as np
    # Your implementation here
    return df
```

(ii) **Dataset information (ψ).** The Dataset Information component provides a comprehensive data profile, grounding the LLM’s reasoning in both statistical metadata and semantic context. To ground the LLM’s reasoning in the dataset’s specific characteristics, we transform the raw data into a structured dataset information block using a multi-level extraction logic:

- **Global profile:** Captures the dataset scale (samples/features), task type (e.g., regression/classification), the numerical-to-categorical ratio, and the semantic meaning of the dataset (if such a description is available), e.g., what the records represent and what the target variable measures.
- **Missingness & quality analysis:** We calculate the missing ratio for every column and flag "high-missing" features that exceed a predefined threshold (e.g., 30%).
- **Heuristic key feature selection:** To manage the LLM’s context window, we score features based on their missingness, presence of domain descriptions, and types. We then prioritize the Top-K most informative features for detailed display.
- **Feature-Level metadata:** Distills specific statistics based on the feature type:
 - Categorical: Includes the number of unique classes and the frequency of the Top-3 categories.
 - Numerical/discrete: Includes statistical moments (mean, standard deviation) and the value range (min, max).

We append human-readable notes (e.g., feature meanings) to the metadata whenever domain knowledge is available.

Template: Dataset Information

[Dataset Profile] {dataset scale, ...}
Target: {...}
All columns: {...}
[Feature Summary] {highlights data quality, specifically identifying columns with high missing-value ratios}
[Key Features] detailed metadata for each feature:
 – col1: type=numerical, missing={%}, mean={ μ }, std={ σ }, range={min, max}, note={description}
 – col2: type=categorical, missing={%}, classes={...}, top={[(val, freq), ...]}, note={description}
 – ... (repeated for prioritized features)

(iii) **Ancestor FE pipeline** (T_{anc}) summarizes the FE pipelines from the original dataset to the currently selected node:

Template: Ancestor FE Pipeline**Current Feature Engineering pipelines:**

→
[Do nothing]
 ↳ Score: {Score₀}
 →
[FE Operation 1]
 ↳ Reason: {Reason₁}
 ↳ Way: {Method₁}
 ↳ Score: {Score₁}
 →
 ...
 →
[FE Operation k]
 ↳ Reason: {Reason_k}
 ↳ Way: {Method_k}
 ↳ Score: {Score_k}

(iv) **Memory of good FE operations** ($\mathcal{M}_{s_{base}}$) shows good feature engineering operations from history that you can reference in the next step:

Template: Memory of Good FE Pipelines**High-performing historical FE operations (memory):**

[Good Operation 1]
 ↳ Reason: {Reason₁}
 ↳ Way: {Method₁}
 ↳ Score: {Score₁}, relative improve: {RelImprove₁}
[Good Operation 2]
 ↳ Reason: {Reason₂}
 ↳ Way: {Method₂}
 ↳ Score: {Score₂}, relative improve: {RelImprove₂}
 ...
[Good Operation m]
 ↳ Reason: {Reason_m}
 ↳ Way: {Method_m}
 ↳ Score: {Score_m}, relative improve: {RelImprove_m}

(v) Directive and Optimization Objectives

Depending on the directive d , the LLM is given optimization objectives that explicitly steer its feature engineering strategy. The directive governs whether the LLM should focus on generating an initial strong operation, exploring novel transformations, or exploiting previously successful experience.

Template: Directive and Objectives**Your general objectives (for any directive d):**

1. Review and summarize the existing feature engineering operations from the original dataset to the currently selected node in the ancestor FE pipeline.
2. Avoid duplicating the feature engineering approaches that have already been attempted in the ancestor FE pipeline.
3. Propose one new feature engineering step. You may reference (but are not limited to) the following categories:
 - Generator: creating new features.
 - Selector: choosing or filtering the most relevant features.
 - Transformation: modifying distributions or encoding categories.
 - Rescaler: normalizing or standardizing features.
 - Imputer: filling missing data via statistical or model-based methods.
4. Your strategy is {**d**} (further specifies the optimization mode and how the above objectives should be prioritized):

Directive: INITIALIZATION (for root node s_0)

Instruction: Propose a **high-quality initial** FE operation for the original dataset. Focus on robust, broadly useful transformations rather than highly specialized or overly complex operations. Aim to establish a strong baseline that later operations can further improve upon.

Directive: EXPLORATION

Instruction: Propose FE operations that **explores new regions** of the transformation space. Prioritize novel or less-explored ideas that are **distinct from existing and previously attempted methods**. Encourage diversity and bold changes to uncover potentially high-performing operations.

Directive: EXPLOITATION

Instruction: Propose FE operations that **refines and exploits** previously successful operations. Try to **reuse, adapt, or combine** high-performing historical FE operations from the memory to further improve performance. Focus on **incremental refinement** rather than broad exploration.